



Laboratorio Linux N° 4
Planificación de Procesos

Linux, al ser un sistema operativo de tiempo compartido (basado en multiprogramación), emplea un algoritmo de planificación de procesos (scheduling) de dos niveles, lo que permite aumentar la cantidad de procesos activos en memoria y ofrecer una pronta respuesta a los procesos interactivos.

El algoritmo de planificación de **alto nivel** se encarga de gestionar el intercambio de procesos entre memoria y disco (conocido como swapping), mientras que el planificador de **bajo nivel** o planificador de CPU selecciona entre los procesos residentes en memoria que están listos para ejecutarse.

Este planificador utiliza **colas multinivel con prioridades**, donde a los procesos de usuario se les asignan valores de prioridad positivos (prioridad menor), y a los procesos del kernel, valores negativos (prioridad mayor). Este sistema permite una diferenciación clara entre procesos críticos para el sistema y aquellos iniciados por el usuario.

El proceso swapper (o sched, dependiendo del contexto y la versión del kernel) tiene la función de decidir qué procesos deben permanecer en memoria y cuáles pueden mantenerse en disco, permitiendo así un equilibrio dinámico. Su tarea es asegurar que todos los procesos tengan la oportunidad de residir en memoria en algún momento, y que ciertos procesos fundamentales (como el propio swapper) permanezcan en memoria de forma permanente.

Cuando múltiples usuarios comparten el sistema, el kernel de Linux se encarga de asignar los recursos de CPU de forma eficiente. Sin embargo, los usuarios tienen cierta capacidad para influir en la planificación del sistema, por ejemplo, ajustando la prioridad de sus propios procesos o programando tareas para que se ejecuten en momentos de baja carga del sistema.

Para ello, Linux provee varias herramientas:

nice: permite iniciar un proceso con una prioridad específica.

renice: modifica la prioridad de un proceso en ejecución.

at: programa la ejecución de un comando en un momento específico.

batch: ejecuta comandos cuando la carga del sistema lo permite.

cron: permite la ejecución periódica de tareas programadas.

Estas herramientas son útiles para optimizar el uso del sistema y evitar que procesos menos urgentes interfieran con tareas críticas o interactivas.

Prioridades de los procesos

El kernel de Linux asigna prioridades iniciales a los procesos de forma automática. Generalmente, los procesos del sistema reciben **prioridades más altas** (es decir, **valores numéricos más bajos o negativos**), mientras que a los procesos que realizan cálculos intensivos o no requieren inmediatez se les asignan **prioridades más bajas**. Esto permite un uso más eficiente y equilibrado de la CPU, privilegiando a los procesos interactivos o críticos.

Cuando un proceso que se estaba ejecutando entra en estado de espera (por ejemplo, esperando un recurso o evento), el sistema selecciona para ejecución, de entre los procesos listos en memoria, **el que tenga mayor prioridad** según la política de planificación.

En cuanto a la gestión de prioridades:

- **Sólo el superusuario (root)** tiene permiso para **incrementar la prioridad** de un proceso (es decir, asignarle un valor más alto en la jerarquía).
- Sin embargo, **cualquier usuario puede reducir la prioridad** de sus propios procesos mediante el comando nice.



EJERCITACIÓN

A. Comprobando la prioridad de los procesos

1. Ejecutá el siguiente comando para ver los procesos actuales y sus prioridades: `$top`. Observá las columnas **PR** (prioridad) y **NI** (nice). Anotá el valor típico que aparece para los procesos de usuario.
2. Salí del top con la tecla q.

B. Ejecutando procesos con diferentes prioridades

1. Ejecutá el siguiente comando sin cambiar la prioridad: `$time find / > /dev/null`. Observá cuánto tarda aproximadamente. *Este comando sirve para testear el rendimiento del sistema de archivos o carga del sistema, sin contaminar la terminal (salvo errores, como por ejemplo cuando no tengo permiso de root para acceder a ciertos directorios)*
2. Ejecutá el mismo proceso con nice, bajando su prioridad: `$time nice -n 15 find / > /dev/null`. ¿Tardó más o menos que el anterior? ¿Por qué?
3. Si tenés acceso como superusuario, ejecutá el mismo comando con **prioridad aumentada**: `$sudo nice -n -10 find / > /dev/null`. ¿Qué diferencias notás?

En caso de muchos errores...

```
Command exited with non-zero status 1
0.82user
5.13system
1:47.16elapsed
5%CPU
(0avgtext+0avgdata 22160maxresident)k
0inputs+0outputs
(0major+7000minor)pagefaults
0swaps
```

C. Cambiando la prioridad de un proceso en ejecución

1. Abrí una terminal nueva y ejecutá un proceso que tarde unos segundos, por ejemplo: `$ yes > /dev/null` (Este comando imprime "yes" infinitamente, pero redirigido al vacío para no llenar la pantalla.)
2. En otra terminal, buscá el PID de ese proceso: `$ ps aux | grep yes` (pueden hacer un `$ps aux` primero para que les queden los encabezados)
3. Cambiá su prioridad: `$ renice +10 -p [PID]` → Reemplazá [PID] por el número real del proceso.
4. Volvé a mirar con `$ top` para ver el cambio en la columna **NI**.
5. Finalizá el proceso con: `$ kill [PID]`



En una pestaña ejecuto:

```
for (( ; ; ))
```

```
do
```

```
continue
```

```
done
```

(termino con Ctrl+C)

Y hago top en el otro y veo como toma toda la CPU.

Si ejecuto lo mismo en otra ventana, igual ocupa el 100

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
310	cecinie+	20	0	6212	5076	3356	R	100.0	0.1	2:57.73	bash
395	cecinie+	20	0	6212	5096	3376	R	100.0	0.1	1:27.86	bash
190	cecinie+	20	0	6212	5204	3428	R	99.0	0.1	6:47.98	bash
438	cecinie+	20	0	6212	5128	3408	R	99.0	0.1	0:38.61	bash
280	cecinie+	20	0	6212	5052	3328	R	96.0	0.1	3:34.63	bash
295	cecinie+	20	0	6212	5084	3364	R	95.7	0.1	3:19.34	bash
367	cecinie+	20	0	6212	5124	3404	R	95.3	0.1	1:51.90	bash
250	cecinie+	20	0	6212	5148	3428	R	93.7	0.1	4:56.84	bash
324	cecinie+	20	0	6212	5080	3364	R	93.7	0.1	2:46.12	bash
410	cecinie+	20	0	6212	5016	3292	R	93.7	0.1	0:59.01	bash
381	cecinie+	20	0	6212	4992	3272	R	93.0	0.1	1:39.90	bash
353	cecinie+	20	0	6212	5052	3332	R	92.7	0.1	2:08.28	bash
466	cecinie+	20	0	6212	5052	3332	R	92.7	0.1	0:16.23	bash
265	cecinie+	20	0	6212	5136	3416	R	91.7	0.1	4:10.65	bash
452	cecinie+	20	0	6212	5160	3436	R	90.7	0.1	0:26.82	bash
338	cecinie+	20	0	6212	5136	3416	R	86.7	0.1	2:32.15	bash
424	cecinie+	20	0	6212	5176	3456	R	86.3	0.1	0:49.11	bash

Puedo crear un archivo para ejecutar varias veces a la vez el mismo bucle de uso ocioso de la CPU:

```
cat > infinite.sh
```

```
for (( ; ; ))
```

```
do
```

```
continue
```

```
done
```

--luego Enter y ctrl+C para armar el archivo.

Cambio permisos para habilitar la ejecución del archivo

```
$chmod +x infinite.sh
```

Luego ejecuto muchos a la vez:

```
for i in {1..NUMERO}
```

```
do /home/user/infinite.sh & --para background
```

```
done
```

Puedo ver el cambio en la consola con \$ top.

Cambio de nice: \$renice -20 ID

Termino la ejecución de todos los procesos asociados al usuario con uno de estos comandos:

```
$pkill -u cecinievas
```

```
$killall -u cecinievas
```

Probar \$top de nuevo.



Programar tareas en Linux

Linux permite ejecutar tareas automáticamente en un momento determinado utilizando el comando `$at`.

at: Ejecutar tareas una sola vez en el futuro

Sirve para programar comandos que se ejecuten **una única vez** en una hora o fecha determinada.

Sintaxis básica:

```
$ at hora  
> comando1  
> comando2  
Ctrl+D
```

Ejercitación

Vamos a programar un mensaje que se guarde en un archivo dentro de **1 minuto**:

```
1.  
$ at now + 1 minute  
at> echo "Hola desde el futuro :)" > ~/mensaje_at.txt  
Ctrl+D
```

Luego hacer cat del archivo.

```
2.  
at 16:45  
at> ls -l > directorio  
at> Ctrl +d
```

```
3.  
at now + 10 minutes  
touch ejemplo.txt  
ctrl + D
```

Otra forma de crear un archivo con contenido: `$echo "contenido del archivo" > archivo.txt`



Alias

Un alias es una forma de crear un atajo o notación abreviada para un comando o una serie de comandos. Nos ayuda a ahorrar tiempo y evitar errores de tipeo.

Sintaxis básica

```
$ alias nombre_alias='comando_largo'
```

Ejemplos

`alias borrar='rm -i'` → Borra con confirmación interactiva (más seguro para evitar errores)

`alias info='uname -a; id; date'` → Muestra info del sistema, usuario y fecha (secuencia de comandos con ;)

Limitaciones:

1. No permite manejo de errores.
2. No se puede usar control de flujo (estructuras de control)

Para mostrar todos los alias creados en el sistema se ejecuta el siguiente comando:

```
$ alias
```

Para eliminar un alias:

```
$ unalias nombre_alias
```

→ A continuación, si desea utilizar el alias borrado, se presentará un error.

Ejercitación

Crear un alias:

```
$ alias hola='echo ¡Hola, mundo!'
```

 Ejecutá hola y verificá el resultado.

Alias con opciones interactivas

```
$ alias borrar='rm -i'
```

```
$ touch prueba.txt
```

```
$ borrar prueba.txt
```

 Observá que pide confirmación antes de borrar.

Alias con múltiples comandos

```
$ alias sistema='uname -a; whoami; date'
```

```
$ sistema
```

→ Creá un alias que combine:

Mostrar contenido de tu carpeta de usuario

Mostrar cuánta memoria está en uso (free -h)

Mostrar los procesos que estás ejecutando (ps u)